Mock Midterm 2, Compilers

Ten MCQs with a trap in each, then every technique once (80 points, 80 minutes)

Compiler Construction Module

Closed book. Alphabet of every automaton is $\{a, b\}$ unless said otherwise. Stack top is on the right.

Summary: How the paper is built

Part A (20 pts): ten multiple-choice questions, 2 pts each. One sentence of justification is required; a letter alone scores 0. Each question contains one of the traps that cost the most points on this material. **Part B (60 pts)**: five problems, one per technique, with no technique repeated: scanning (B1), automata (B2), grammar design and ambiguity (B3), the LL(1) pipeline (B4), recursive descent (B5).

Part A: Multiple choice with justification (20 pts)

Problem: A1 (2 pts): which phase complains?

The source line is `x = 3y + 1;` in a C-like language whose scanner has the usual rules for identifiers, integers, operators and whitespace. Which phase reports the first problem?
(A) The scanner: `3y` is not a valid number. (B) The parser: the token sequence is wrong. (C) The semantic analyser: `y` is undeclared. (D) No phase: the line is legal.

Problem: A2 (2 pts): longest match versus rule order

A scanner has, in this order, rule 1 `if` $\to$ IF, rule 2 `[a-z]+` $\to$ ID, rule 3 `[a-z]+[0-9]+` $\to$ TAG. The input is `if9`. What is returned?
(A) IF, then NUM(9). (B) ID(`if`), then NUM(9). (C) TAG(`if9`). (D) A lexical error at `9`.

Problem: A3 (2 pts): counting Thompson's construction

Using the lecture's construction (every union and every star adds two new states and four ε-edges; concatenation merges the final state of the left part with the start state of the right part), how many states and edges does the NFA for `(a|b)*` have?
(A) 6 states, 8 edges. (B) 8 states, 10 edges. (C) 8 states, 8 edges. (D) 4 states, 6 edges.

Problem: A4 (2 pts): how big is the DFA?

Let L be the strings over $\{a, b\}$ whose third symbol from the end is an a, i.e. `(a|b)*a(a|b)(a|b)`. How many states does the minimal DFA for L have?
(A) 4. (B) 5. (C) 8. (D) 16.

Problem: A5 (2 pts): which one is regular?

Exactly one of these languages over $\{a, b\}$ is regular. Which?
(A) $\{a^n b^n : n \geq 0\}$. (B) Strings in which the substring `ab` occurs as many times as the substring `ba`. (C) $\{a^n b^m a^n : n, m \geq 0\}$. (D) $\{ww : w \in \{a, b\}^*\}$.

Problem: A6 (2 pts): is ε in FIRST?

$T \to F\,T'$, $T' \to *\,F\,T' \mid \varepsilon$, $F \to$ `(` E `)` $\mid$ `id`. Which is true?
(A) $\varepsilon \in \text{FIRST}(T)$ because T' is nullable. (B) $\varepsilon \notin \text{FIRST}(T)$ because F is not nullable. (C) $\varepsilon \in \text{FIRST}(T)$ because T has two symbols on its right side. (D) $\varepsilon \in \text{FIRST}(F)$ because E may be nullable.

Problem: A7 (2 pts): FOLLOW through a nullable neighbour

$S \to A\,B$, $A \to a \mid \varepsilon$, $B \to C\,d$, $C \to c \mid \varepsilon$. What is $\text{FOLLOW}(A)$?
(A) $\{c\}$. (B) $\{c, d\}$. (C) $\{d\}$. (D) $\{\$\}$.

Problem: A8 (2 pts): the dangling else

$S \to$ if E then $S \mid$ if E then S else $S \mid$ other. Which statement is true?
(A) Left factoring makes the grammar LL(1). (B) After left factoring, $M[S', \text{else}]$ still holds two productions; no rewriting of the grammar removes the ambiguity, and practical parsers resolve it by always taking the else branch. (C) The grammar is unambiguous but left-recursive. (D) The language is not context-free.

Problem: A9 (2 pts): an empty cell

An LL(1) parser has E on top of the stack and the next input token is `)`. The table cell $M[E,)]$ is empty, and `)` $\in \text{FOLLOW}(E)$. What does the parser do?
(A) Pops E and continues, because `)` is in $\text{FOLLOW}(E)$. (B) Applies $E \to \varepsilon$. (C) Reports a syntax error. (D) Skips the token `)` and retries.

Problem: A10 (2 pts): Chomsky type

$S \to a\,S\,b \mid a\,b$. Which statement is correct?
(A) The grammar is regular because every right side contains at most one nonterminal. (B) The grammar is context-free, and its language is not regular. (C) The grammar is context-sensitive because terminals appear on both sides of the nonterminal. (D) The grammar is context-free and its language is regular.

Part B: One problem per technique (60 pts)

Problem: B1 (12 pts): scanning with backup

In the toy language, an *identifier* starts with a lowercase letter, continues with lowercase letters, digits and underscores, but two underscores may never be adjacent and an underscore may not be the last character. A *number* is `[0-9]+(\.[0-9]+)?([eE][+-]?[0-9]+)?`. Blanks are skipped. When no rule matches at the current position, the scanner reports that one character as an error

and skips it.

(a) Write a regular expression for identifiers. [3 pts]

(b) Draw a DFA for identifiers directly from the description (no Thompson, no subset construction). Say what each state remembers. [4 pts]

(c) Give the exact sequence of tokens and errors the scanner produces for the line `ab__c1 2.e5 x_`, and for every token that required *backing up*, say how many characters were read beyond the lexeme. [5 pts]

Problem: B2 (12 pts): subset construction and minimisation

The NFA N has states $0, 1, 2, 3$, start state 0, the single accepting state 0, and the edges

$$0 \xrightarrow{a} 2, \quad 0 \xrightarrow{b} 1, \quad 1 \xrightarrow{b} 2, \quad 1 \xrightarrow{b} 3, \quad 2 \xrightarrow{a} 0, \quad 2 \xrightarrow{a} 3, \quad 2 \xrightarrow{\varepsilon} 3.$$

(a) Give ε-closure($\{0\}$) and the DFA's start state. [2 pts]

(b) Run the subset construction. Give the full transition table, including the dead state if one is needed, and mark the accepting states. [5 pts]

(c) Minimise the DFA with the table-filling method. Show which pairs get marked in each sweep, and the resulting minimal DFA. [4 pts]

(d) Give a regular expression for $L(N)$. [1 pts]

Problem: B3 (12 pts): designing a grammar, proving and removing ambiguity

(a) Give a context-free grammar for $L = \{a^n\, b^m\, c^{n+m} : n, m \geq 0\}$ and explain in two sentences why it generates exactly L. [4 pts]

(b) Show that $E \to E$ `?` E `:` $E \mid E$ `+` $E \mid$ `id` is ambiguous, using the sentence `id + id ? id : id`. [3 pts]

(c) Write an unambiguous grammar for the same language in which `+` is left-associative and binds tighter than the conditional `? :`, and the conditional is right-associative (`a ? b : c ? d : e` means `a ? b : (c ? d : e)`). Draw the single tree of `id ? id : id ? id : id`. [5 pts]

Problem: B4 (14 pts): the LL(1) pipeline, once

Grammar G, start symbol *Stmt*, terminals `id num = ( ) , ;`:

1	*Stmt* $\to$ `id =` *Exp* `;`	4	*More* $\to$ `,` *Exp More* $\mid \varepsilon$
2	*Stmt* $\to$ `id (` *Args* `) ;`	5	*Exp* $\to$ `id` $\mid$ `num`
3	*Args* $\to$ *Exp More* $\mid \varepsilon$		

(a) G is not LL(1). Name the exact table cell and the reason, and say which transformation fixes it (it is not left recursion). Write the transformed grammar G' with numbered productions. [3 pts]

(b) For G': the nullable nonterminals, then FIRST and FOLLOW of every nonterminal. [4 pts]

(c) The LL(1) table of G' (production numbers in the cells). [3 pts]

(d) The stack trace (stack, remaining input, action) for `id ( id , num ) ;`. [4 pts]

Problem: B5 (10 pts): a recursive-descent parser with two bugs

A student wrote this parser for expressions with `+`, identifiers and parentheses. `la` is the lookahead, `match(t)` consumes `t` or calls `error()`.

```
void E()  { T(); Ep(); }
void Ep() { if (la == '+')      { match('+'); T(); Ep(); }
            else if (la == ')') { match(')'); }
            else { } }
void T()  { if (la == ID)       match(ID);
            else if (la == '(') { match('('); E(); match(')'); }
            else error(); }
void parse() { la = nextToken(); E(); }
```

(a) Write the grammar the student *intended* (the usual one), with its selection sets. [2 pts]

(b) Trace the parser on the valid input `( ID )` and show where and why it fails. [3 pts]

(c) Fix `Ep()` and justify the fix with a FOLLOW set. [2 pts]

(d) With `Ep()` fixed, the parser still accepts the invalid input `ID ID`. Trace it, name the missing check, and say where the equivalent check lives in the table-driven stack parser. [3 pts]